Алматинский Технологический Университет

Кафедра информационных систем

Кэш расписания университета (Redis + MongoDB)

СРО по дисциплине «Проектирование и применение NoSQL-технологий», вариант 11

Выполнил: Евгений Л, группа ИС 24-21

Преподаватель: Рашидов Дамир

2026

Проблема и цель

Проблема

- расписание группы студенты открывают по много раз в день, особенно утром и на переменах
- собирается оно из 4 коллекций (группы, преподаватели, аудитории, занятия) через $lookup
- меняется редко, но после переноса пары старую аудиторию показывать нельзя

Цель

- MongoDB хранит расписание, Redis отдает готовое по ключу
- после изменения пары кэш сбрасывается (TTL + удаление ключа)
- измерить, сколько живут устаревшие данные в разных вариантах

Требования и access patterns

Функциональные: расписание группы (неделя, день), преподавателя, аудитории; добавить, перенести, удалить пару; после изменения следующий запрос свежий; свободные аудитории, нагрузка, конфликты; статистика кэша.

Нефункциональные: повторное чтение быстрее 1 мс; stale data 0 в штатном режиме и не больше TTL в худшем; кэш ничего не хранит эксклюзивно; запуск через docker compose, без секретов.

AP	Запрос	Частота	Где
1-2	группа: неделя / день	очень часто	Redis
3	преподаватель	часто	Redis
4	аудитория	средне	Redis
5-6	перенос / добавить / удалить	редко	Mongo + DEL
7	свободные аудитории	редко	Mongo $lookup
8	нагрузка преподавателей	раз в семестр	Mongo $group
9	конфликты аудиторий	после правок	Mongo $group
10	поиск по предмету	редко	Mongo regex

Выбор технологии (ADR)

Вариант	Плюсы	Минусы
Только MongoDB	одна система, всегда свежо	$lookup из 4 коллекций на каждое открытие страницы
Только Redis	очень быстро	нет сложных запросов, плоские данные
PostgreSQL + кэш	привычные JOIN	кэш все равно нужен, жесткая схема не нужна
MongoDB + Redis (cache-aside)	Mongo хранит и считает аналитику, Redis отдает готовое за доли мс	две системы, за согласованностью кэша следит приложение

Write-through не стал делать: при переносе одной пары пришлось бы пересобирать до 6 расписаний, а читают их не сразу. Проще удалить ключ и собрать по первому запросу.

Модель данных

MongoDB, база university

```
groups   { code, course, faculty, students }
teachers { teacher_id, name, department }
rooms    { room, building, capacity, type }
lessons  { lesson_id, group_code, subject,
           teacher_id, room, weekday,
           start, end, kind }
```

Ссылки по group_code, teacher_id, room. Индексы (group_code, weekday, start), (teacher_id, weekday), (room, weekday, start). Данные: 6 групп, 8 преподавателей, 7 аудиторий, 40 занятий.

Redis

Ключ	Значение	TTL
schedule:group:IS-31	JSON недели	300 с
schedule:teacher:2	JSON недели	300 с
schedule:room:B-201	JSON недели	300 с
stats:hits / misses / invalidations	счетчики	нет

В JSON уже есть имя преподавателя и корпус, при попадании в Mongo не ходим. День фильтруется из недели, своего ключа нет.

Архитектура (cache-aside)



Чтение

- GET schedule:group:IS-31
- нет - aggregate в Mongo
- SET key EX 300

Запись

- update lessons в Mongo
- DEL ключей группы, преподавателя, старой и новой аудитории

Реализация

```
def _cached(self, key, loader):
    data = self.cache.get(key)
    source = "cache"
    if data is None:
        data = loader()         # mongo
        self.cache.set(key, data)
        source = "mongo"
    return data, source

def change_lesson(self, id, changes):
    old = self.repo.update_lesson(id, changes)
    new = self.repo.get_lesson(id)
    return self._drop(old, new)  # DEL
```

Python 3.9, pymongo, redis, pytest, docker compose

- src/schedule_cache: repository (Mongo), cache (Redis), service (cache-aside)
- scripts: seed, CLI, demo_queries, stale_analysis
- tests: 14 тестов, отдельная база и Redis db 1
- docs: ADR, схема, отчет

Запросы: перенос пары и инвалидация

```
$ python scripts/schedule.py group IS-31          -> mongo, 8.44 ms, 8 занятий
$ python scripts/schedule.py group IS-31          -> cache, 0.51 ms, ttl 300 с
$ python scripts/schedule.py move L001 --room B-201
  ключи до:    schedule:group:IS-31, schedule:room:B-201, schedule:teacher:2
  сброшены:    schedule:group:IS-31, schedule:teacher:2, schedule:room:B-202, schedule:room:B-201
  ключи после: []
$ python scripts/schedule.py group IS-31          -> mongo, L001 теперь в B-201
```

- Свободные аудитории Чт 14:30 ($lookup с pipeline): 5 из 7, заняты A-101 и B-202
- Нагрузка преподавателей ($group + $lookup): Сериков 7 пар / 9.3 ч, Касымова 2 пары / 2.7 ч
- Конфликты ($group по аудитория+день+время): B-202 Чт 14:30, группы IS-31 и IT-41

Stale data и тесты

Сценарий	Кэш отдавал старую аудиторию
1. Только TTL=5 с, ключ не удаляем	4.89 с, до истечения TTL
2. TTL=300 с + DEL после изменения	**0.00 с**
3. Гонка: читатель положил старое после DEL	4.88 с, спас только TTL
4. 300 случайных чтений, 6 групп	hit ratio 0.98, Redis 0.34 мс, Mongo 3.69 мс

- без DEL студент видит старую аудиторию весь TTL (5 минут), с DEL окно нулевое
- гонка остается, поэтому TTL короткий и работает как страховка
- 14 тестов pytest: miss/hit, истечение TTL, сброс ключей старой и новой аудитории, stale без инвалидации, CRUD, свободные аудитории, конфликты

Ограничения и вывод

Ограничения

- инвалидацию делает приложение, правка в mongosh мимо сервиса живет в кэше до TTL
- гонка чтения и записи не закрыта (можно повторным DEL через секунду или версией в ключе)
- один Redis и один Mongo без репликации, нет недель числитель/знаменатель

Вывод

Cache-aside с удалением ключей после изменения дает нулевое окно устаревания в штатном режиме, короткий TTL страхует от гонки. 98% чтений уходят в Redis и отвечают в 10 раз быстрее, чем сборка в Mongo.